环境搭建

使用 `conda create -n derain python=3.10` 创建一个新的环境，并在requirement所在目录下用 `pip install -r requirements.txt` 安装任务所需要的依赖，嫌慢可以用国内镜像源，如通过阿里云下载 `pip install -r requirements.txt -i https://mirrors.aliyun.com/pypi/simple/` （实测速度差不多）

安装完成后测试是否安装成功：

```
(base) pigeon@LAPTOP-GARN0974:~/ml-week$ conda activate derain
(derain) pigeon@LAPTOP-GARN0974:~/ml-week$ python -c "import torch, torchvision, numpy, PIL, skimage, tqdm, lpips; print(torch.__version__, torchvision.__version__, torch.cuda.is_available())"
2.4.1+cpu 0.19.1+cpu False
(derain) pigeon@LAPTOP-GARN0974:~/ml-week$
```

实现一个简单的模型

1.模型代码：

```
import torch.nn as nn

class Block(nn.Module) :
    def __init__(self, in_channels, out_channels) :
        super(Block, self).__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(in_channels, out_channels, 3, 1, 1, bias = False),
            nn.InstanceNorm2d(out_channels),
            nn.ReLU(inplace = True),
            nn.Conv2d(out_channels, out_channels, 3, 1, 1, bias = False),
            nn.InstanceNorm2d(out_channels),
        )
        self.relu = nn.ReLU(inplace = False)

    def forward(self, x) :
        return self.relu(x + self.conv(x))

class BaselineNet(nn.Module):
    def __init__(self, in_channels = 3, out_channels = 3, channels = 64, num = 4):
        super(BaselineNet, self).__init__()
        self.blocks = nn.ModuleList()
        self.entry = nn.Conv2d(in_channels, channels, 3, 1, 1)
        for i in range(num) :
            self.blocks.append(Block(channels, channels))
        self.exit = nn.Conv2d(channels, out_channels, 3, 1, 1)

    def forward(self, x):
        s = x
        x = self.entry(x)
        for block in self.blocks:
            x = block(x)
        x = self.exit(x)
        return s - x
```

2.针对这个问题，我们可以从不同角度解决：

解决堆叠非常深的网络才能让像素间进行远距离信息交流的问题，即降低所需的深度，考虑以下几种较易实现的方法：

①使用空洞卷积，让卷积核中每个参数之间间隔 d 个像素，在不增大参数量的情况下增大感受野。

②学习unet.py中的下采样+用转置卷积上采样的做法，下采样把通过降低图像分辨率的方法将图像“缩小”来增大感受野，下采样的分辨率再由一步步乘上转置矩阵来增大回来。

另外，我们还可以解决深层网络难以训练的问题，最简单的方法就是使用残差连接，把某一层的输出直接加到后面若干层的输出上，这样反向传播的时候梯度就可以直接通过这条路传播回去，从而缓解梯度消失。另外网络中本身存在的Norm归一化、训练时选择合适的优化器和调度器也能缓解深层网络难以训练的问题。

3. 这个代码是什么意思？

1.main.py中设置命令行参数的函数为 `get_args`：

```
def get_args():
    parser = argparse.ArgumentParser(description="Deraining Model Training")
    parser.add_argument("--data_dir", type=str, required=True, help="Path to the
dataset")
    parser.add_argument("--model", type=str, default="baseline", choices=
["baseline", "unet"], help="Model to use")
    parser.add_argument("--mode", type=str, default="train", choices=["train",
"test"], help="Train or test mode")
    parser.add_argument("--checkpoint", type=str, default=None, help="Path to
checkpoint for testing")

    parser.add_argument("--epochs", type=int, default=20, help="Number of training
epochs")
    parser.add_argument("--batch_size", type=int, default=8, help="Batch size")
    parser.add_argument("--lr", type=float, default=1e-4, help="Learning rate")

    parser.add_argument("--use_perceptual_loss", action="store_true", help="Use
perceptual loss")
    parser.add_argument("--lambda_pixel", type=float, default=1.0, help="Weight
for pixel loss")
    parser.add_argument("--lambda_perceptual", type=float, default=0.1,
help="Weight for perceptual loss")

    return parser.parse_args()
```

其中使用 `argparse.ArgumentParser()` 创建参数解析器，并使用 `parser.add_argument()` 来设定参数，其中 `type` 表示参数类型，`required=True` 表示这个参数是必填项，`default` 表示默认值，`choices` 表示可选的参数值。

测试结果：

```
(derain) pigeon@LAPTOP-GARN0974:~/ml-week/Image-Deraining-Recruitment-Task$ python main.py --data_dir "./data/Rain200L" --model baseline --mode test --checkpoint
 "checkpoints/best_baseline.pth.tar"
  warnings.warn(
/home/pigeon/miniconda3/envs/derain/lib/python3.10/site-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or `None` for
'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equivalent to passing `weights=AlexNet_Weights.IMAGENET1K_V1`. You
can also use `weights=AlexNet_Weights.DEFAULT` to get the most up-to-date weights.
  warnings.warn(msg)
Downloading: "https://download.pytorch.org/models/alexnet-owt-7be5be79.pth" to /home/pigeon/ml-week/.cache/torch/hub/checkpoints/alexnet-owt-7be5be79.pth
100%|████████████████████████████████████████████████████████████████████████████████████████████████████████| 233M/233M [00:02<00:00, 120MB/s]
Loading model from: /home/pigeon/miniconda3/envs/derain/lib/python3.10/site-packages/lpips/weights/v0.1/alex.pth
/home/pigeon/miniconda3/envs/derain/lib/python3.10/site-packages/lpips/lpips.py:107: FutureWarning: You are using `torch.load` with `weights_only=False` (the cur
rent default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during
unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for `weights_
only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via
 this mode unless they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for
any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  self.load_state_dict(torch.load(model_path, map_location='cpu'), strict=False)
/home/pigeon/ml-week/Image-Deraining-Recruitment-Task/main.py:123: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value
), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See ht
tps://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for `weights_only` will be flip
ped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless
they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case where
 you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  load_checkpoint(torch.load(args.checkpoint, map_location=device), model, optimizer)
=> Loading checkpoint
EVAL => Avg PSNR: 33.1598 | Avg SSIM: 0.9450 | Avg LPIPS: 0.0290
```

2.从代码中学到的新东西：

from `main.py`：学习到了设置参数解析器、添加参数的方法、学到了使用 `tqdm` 包装 `dataloader` 显示进度条、用 `to(device)` 把数据搬到cpu/gpu上计算。另外，应该把 `train_one_epoch` 和 `evaluate` 封装为函数，每次epoch调用而不是直接放在主函数里。

from `unet.py`：`nn.InstanceNorm2d` 不同于 `BatchNorm`， `InstanceNorm` 中的平均值和方差是每张图独立算出的而不是一批图整体算出的，因此使用 `InstanceNorm` 的时候图像不会受到同批次的其他图像的影响，使训练结果更稳定，对于图片修复这种工作来说能够保留不同样本之间的独特性；`nn.ModuleList()`，专门用于存放pytorch中子模块的list，可以整合子模块，在调用、访问的时候更方便；`nn.ConvTranspose2d()`，转置卷积，用于上采样恢复图像的分辨率。

from `utils.py`：调用库计算psnr、ssim、lpips；`next` 取用迭代器的下一个元素，用于取第一个batch。

3.已替换：

```
import math
import torch.nn as nn

def calc_psnr(img1, img2, data_range : float = 1.0) -> float :
    img1 = img1.detach().clamp(0, data_range)
    img2 = img2.detach().clamp(0, data_range)

    mses = nn.MSELoss(reduction = "none")(img1, img2)
    mses = mses.reshape(mse.shape[0], -1).mean(dim = 1)
    psnr_list = []

    for mse in mses.tolist() :
        if abs(mse) < 1e-5 :
            psnr_list.append(100.0)
        else :
            psnr_list.append(10.0 * math.log10(data_range ** 2 / mse))
    return sum(psnr_list) / len(psnr_list)

#================================================================
```

```
def calculate_metrics(img1, img2, lpips_fn):
    avg_psnr = calc_psnr(img1, img2, data_range = 1.0)
    img1_np = img1.detach().cpu().numpy().transpose(0, 2, 3, 1)
    img2_np = img2.detach().cpu().numpy().transpose(0, 2, 3, 1)
    img1_np = np.clip(img1_np, 0, 1)
    img2_np = np.clip(img2_np, 0, 1)

    batch_ssim = 0
    for i in range(img1_np.shape[0]):
        # batch_psnr += psnr(img1_np[i], img2_np[i], data_range=1.0)
        batch_ssim += ssim(img1_np[i], img2_np[i], data_range=1.0,
multichannel=True, channel_axis=2, win_size=7)

    # avg_psnr = batch_psnr / img1_np.shape[0]
    avg_ssim = batch_ssim / img1_np.shape[0]

    img1_lpips = img1.clone() * 2 - 1
    img2_lpips = img2.clone() * 2 - 1

    with torch.no_grad():
        lpips_score = lpips_fn(img1_lpips, img2_lpips).mean().item()

    return avg_psnr, avg_ssim, lpips_score
```